SOFTWARE DESIGN PATTERNS

SmartHall Case Study

Command, Adapter & Facade: Group Homework, Marking Rubric and Group Variants

Contents	Page
Part A: Case Study and Milestones	2
Part B: Marking Rubric and Group Marking Sheet	8
Appendix A: Group Variant Sheets (Groups 1 to 16)	12

PART A

Case Study and Milestones

Command, Adapter & Facade (Head First Design Patterns, Chapters 6 and 7). The marks for each milestone are shown in its heading. The marking rubric is in **Part B**, and your group's own variant sheet is in **Appendix A**.

Item	Details
Patterns	Command (with undo, macros, logging), Adapter, Facade, plus the Principle of Least Knowledge
Groups	16 groups. Each group has a **unique variant** (see the Variant Table), so no two groups submit the same solution
Language	Java 17 or newer, plain Java (no frameworks needed). A console test harness is enough; no GUI required
Total marks	100 marks, plus 10 optional stretch marks
Due	_____ (live design defence in the following lab session)

1. The Story

The university has refurbished its lecture theatres. Over ten years the Estates Office bought lights, a PA system, a motorised screen, air conditioning, blinds and a lecture-capture recorder from **six different vendors**, and no two of them share an interface. Lecturers currently need a printed checklist and about four minutes to set a hall up, and they often forget to switch things off afterwards.

Estates has commissioned **SmartHall**: a wall control panel with programmable On/Off slots and an Undo button, one-touch **modes** (such as “Exam Mode”), an audit log of every action, and a simple API for the timetabling system: `startLecture("CSC 2201")` and `endLecture()`. To complicate things, every hall has just received a **new projector from a vendor whose API doesn't match** the `Projector` interface the existing SmartHall code already uses.

Your group is the software team. Your job maps directly onto Chapters 6 and 7: **Command** for the wall panel, **Adapter** for the new projector, and **Facade** for the timetabling API.

2. Starter Code (Given, Do Not Modify the Vendor Classes)

Implement each vendor method body as a simple `System.out.println` that reports what the device did, as the book does. You may add a `location` constructor argument.

JAVA: STARTER CODE

```java
// ---- Vendor classes: you may NOT change these method signatures ----
public class HallLights      { void on(); void off(); void dim(int percent); int getLevel(); }
public class PASystem        { void powerOn(); void powerOff(); void setVolume(int level);
                               int getVolume(); void mute(); void unmute(); }
public class MotorisedScreen { void lower(); void raise(); boolean isLowered(); }
public class AirConditioner  { void start(); void stop(); void setTemperature(int celsius);
                               int getTemperature(); }
public class Blinds          { void open(); void close(); }
public class LectureRecorder { void startRecording(String courseCode); void stopRecording();
                               boolean isRecording(); }

// ---- Existing SmartHall interface: older code already depends on it ----
public interface Projector {
    void on();
    void off();
    void setSource(String source);   // "HDMI1", "HDMI2" or "WIRELESS"
    void setKeystone(int degrees);   // keystone correction, -15..15
}
```

```java
// ---- Existing test helper: your adapter must pass through it unchanged ----
public class ProjectorCheck {
    public static void check(Projector p) {
        p.on(); p.setSource("HDMI1"); p.setSource("WIRELESS"); p.off();
    }
}
```

Your new projector is set by your variant. Its class can't be modified, and its method names, parameters and input codes differ from `Projector`. For example, the **VisioMax** variant:

JAVA: EXAMPLE ADAPTEE (VARIANT V)

```java
// Vendor: VisioMax. New 4K projector, closed source.
public class VisioMaxProjector {
    public void switchOn(int warmUpSeconds);       // must be >= 5
    public void switchOff(boolean runCoolingFan);  // always true in a lecture hall
    public void selectInput(int port);             // 1 = HDMI1, 2 = HDMI2, 7 = WIRELESS
    // NOTE: no keystone support at all
}
```

3. Milestones

Milestone 1: The Wall Panel (Command basics), 20 marks

1. Define `Command` with `execute()` and `undo()`.
2. Write at least **eight** concrete commands covering at least **five** of the vendor devices (e.g. `LightsDimCommand`, `PAVolumeCommand`, `BlindsCloseCommand`).
3. Build the `WallPanel` invoker with your variant's number of slots, `setCommand(slot, on, off)`, `onPressed(slot)`, `offPressed(slot)`, and a readable `toString()`.
4. Use a **null object** (`NoCommand`) so the panel never checks for null.
5. Write a `PanelLoader` test harness (the Client) that loads the slots and presses every button.

Design question (answer in at most 120 words): Name the Client, Invoker, Command and Receiver in your code, and explain why `WallPanel` never imports a vendor class.

Milestone 2: Undo That Remembers (stateful undo), 20 marks

1. Every command implements `undo()`.
2. `PAVolumeCommand`, `LightsDimCommand` and `ACTemperatureCommand` must restore the **previous** level, volume or temperature, as `CeilingFanHighCommand` does with `prevSpeed`.
3. Replace the single `undoCommand` with a **history stack** (`Deque<Command>`) capped at your variant's undo depth. When the stack is full, the oldest entry is dropped.
4. Show a test that presses at least six buttons, then presses Undo until the history is empty, and prints the device state after each step.

Design question (at most 120 words): Why must a stateful command capture its state inside `execute()` and not in its constructor? Give a concrete failure case from your hall.

Milestone 3: Modes and the Audit Log (macros and logging), 15 marks

1. Implement `MacroCommand`, then build your variant's **mode** (see the Mode Table) as an On macro and a matching Off macro.
2. Macro `undo()` must reverse every step in **reverse order**, and so must undoing a mode from the history stack.

3. Add an `AuditLog` that records `timestamp, slot, command class` for every executed command. Simulate a crash by creating fresh devices, then **replay** the log to restore the hall's state.

Design question (at most 120 words): Your mode turns the PA on and then sets its volume. What would go wrong if `undo()` ran forward instead of backward?

Milestone 4: The New Projector (Adapter), 20 marks

1. Write `<Vendor>ProjectorAdapter implements Projector` as an **object adapter**: it composes your variant's projector class.
2. Translate each method: `on()`, `off()`, and `setSource(String)` mapped to your variant's input codes. An unknown source throws `IllegalArgumentException`.
3. Your vendor has no keystone support, so `setKeystone()` must throw `UnsupportedOperationException`, and the class's Javadoc must say so, just as `EnumerationIterator.remove()` does.
4. Prove it works: `ProjectorCheck.check(new <Vendor>ProjectorAdapter(...))` runs **without any change** to `ProjectorCheck`.

Design question (at most 150 words): Could you have written a class adapter here? Explain what Java would and wouldn't let you do, and which you'd choose if the vendor later released a `VisioMaxProProjector` subclass.

Milestone 5: The Timetabling API (Facade and Least Knowledge), 25 marks

1. Write `LectureHallFacade`, composed of all the hall's devices **including your adapted projector**, with `startLecture(String courseCode)`, `endLecture()`, `startMode()` and `endMode()` for your variant's mode.
2. `startLecture()` must handle ordering that a lecturer forgets: projector warm-up before selecting a source, PA on before setting volume, recorder started last.
3. **Combine the patterns:** add `StartLectureCommand` and `EndLectureCommand`, whose **receiver is the facade**, and load them into a `WallPanel` slot. One button now runs the whole lecture setup, and it can be undone.
4. **Least Knowledge:** the dashboard code below violates the principle. Rewrite it, and add whatever method(s) are needed so it talks only to its friends:

```
JAVA: FIX THIS
public class CampusDashboard {
    Building building;
    public int hallVolume(String hallId) {
        return building.getHall(hallId).getFacade().getPASystem().getVolume();
    }
}
```

Design question (at most 150 words): Your mode now exists twice, as a `MacroCommand` (Milestone 3) and as a facade method. When should a one-touch action be a macro, and when a facade method? Use your hall as the example.

4. Variant Table (One Row per Group)

Each group has its own projector vendor, input codes, mode, panel size and undo depth. Copying another group's code won't produce a correct solution for yours. Appendix A gives each group a one-page sheet with its full vendor class and mapping.

Group	Projector vendor (adaptee)	Input codes	Mode	Panel slots	Undo depth
1	VisioMax: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`	HDMI1=1, HDMI2=2, WIRELESS=7	Exam Mode	6	5
2	LumaBeam: `powerUp()`, `powerDown()`, `routeTo(char)`	HDMI1='A', HDMI2='B', WIRELESS='W'	Guest Lecture Mode	7	4
3	ClearCast: `start()`, `shutdown()`, `input(String)`	HDMI1="H1", HDMI2="H2", WIRELESS="CAST"	Video Screening Mode	5	6
4	PixelForge: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`	HDMI1=10, HDMI2=11, WIRELESS=20	Night Class Mode	8	3
5	VisioMax: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`	HDMI1=3, HDMI2=4, WIRELESS=9	Power-Save Mode	7	8
6	LumaBeam: `powerUp()`, `powerDown()`, `routeTo(char)`	HDMI1='X', HDMI2='Y', WIRELESS='Z'	Seminar Mode	6	5
7	ClearCast: `start()`, `shutdown()`, `input(String)`	HDMI1="IN-1", HDMI2="IN-2", WIRELESS="AIR"	Lab Demo Mode	8	7
8	PixelForge: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`	HDMI1=1, HDMI2=2, WIRELESS=99	Viva Mode	5	4
9	VisioMax: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`	HDMI1=5, HDMI2=6, WIRELESS=8	Guest Lecture Mode	8	10
10	LumaBeam: `powerUp()`, `powerDown()`, `routeTo(char)`	HDMI1='1', HDMI2='2', WIRELESS='R'	Exam Mode	5	6
11	ClearCast: `start()`, `shutdown()`, `input(String)`	HDMI1="A", HDMI2="B", WIRELESS="MIRROR"	Night Class Mode	7	3
12	PixelForge: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`	HDMI1=30, HDMI2=31, WIRELESS=40	Video Screening Mode	6	9
13	VisioMax: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`	HDMI1=2, HDMI2=1, WIRELESS=0	Seminar Mode	5	5
14	LumaBeam: `powerUp()`, `powerDown()`, `routeTo(char)`	HDMI1='P', HDMI2='Q', WIRELESS='S'	Power-Save Mode	8	4
15	ClearCast: `start()`, `shutdown()`, `input(String)`	HDMI1="PORT_A", HDMI2="PORT_B", WIRELESS="WIFI"	Viva Mode	6	8
16	PixelForge: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`	HDMI1=7, HDMI2=8, WIRELESS=15	Lab Demo Mode	7	6

Vendor notes: VisioMax's `switchOn` needs a warm-up of at least 5 seconds. LumaBeam's `routeTo` silently ignores unknown characters, so your adapter must reject them itself. ClearCast's `input` codes are case-sensitive. PixelForge's `activate(true)` enables eco mode; lecture halls use `false`. **No vendor supports keystone.**

5. Mode Table

Mode	On macro steps (in this order)	Off macro
Exam Mode	Lights 100% → PA mute → blinds open → recorder stop → AC 21 °C → screen up	Reverse it, or restore the previous state

Mode	On macro steps (in this order)	Off macro
Guest Lecture Mode	Lights 70% → screen down → projector on, HDMI2 → PA on, volume 6 → AC 23 °C → recorder start	Recorder stop, then shut down
Video Screening Mode	Blinds close → lights 10% → screen down → projector on, HDMI1 → PA on, volume 8	Lights 100%, devices off, blinds open
Night Class Mode	Lights 100% → blinds close → AC 24 °C → PA on, volume 5 → projector on, WIRELESS	All off; blinds stay closed
Power-Save Mode	Projector off → PA off → AC off → screen up → lights off	Restore the previous state of every device
Seminar Mode	Blinds open → lights 80% → PA on, volume 4 → projector on, WIRELESS → recorder start	Recorder stop, then shut down
Lab Demo Mode	Lights 60% → screen down → projector on, HDMI2 → PA on, volume 7 → AC 22 °C	Reverse it
Viva Mode	Blinds close → lights 90% → PA on, volume 3 → recorder start → screen up	Recorder stop, PA off, lights 100%

6. Academic Integrity and AI Use

- **Design answers carry about a fifth of the marks.** Write them in your own words about your hall; generic textbook answers score zero.
- **Live defence (5 minutes per group).** In the lab, any member may be asked to explain one class and make a small live change. For example: “Add a BlindsOpenCommand to slot 2 and show undo working.”
- **Show your process.** Submit your git history. A single “final commit” with no earlier work will be queried.
- **AI disclosure.** If you used an AI assistant, add a short note saying what you asked and what you changed. Undisclosed use counts as an integrity breach.

7. Stretch Goal (+10): Two-Way Attendance Adapter

The legacy roll-call system exposes `tick(String regNo, int dayOfYear)` and `boolean wasTicked(String regNo, int dayOfYear)`. The new SmartHall code expects `AttendanceTracker` with `markPresent(Student s, LocalDate d)` and `isPresent(Student s, LocalDate d)`. Write **one** class that implements **both** interfaces (a two-way adapter, as described on p. 244), so old and new code can share a single register. `startLecture()` should mark the lecturer present.

8. Submission Checklist

- Source code (one package per milestone, or a single package with clear class names)
- Console output from each milestone's test harness, saved as text
- One UML class diagram covering all five milestones, with Client, Invoker, Command, Receiver, Target, Adapter, Adaptee and Facade labelled
- Design answers (Milestones 1–5) in one PDF, with the group number and variant row at the top
- Git history export, and an AI-use note if applicable

PART B

Marking Rubric

SmartHall Case Study: Marking Rubric

Command, Adapter & Facade group homework. 100 marks, plus 10 optional stretch marks. Use the summary to check totals, the detailed criteria to decide each score, and the marking sheet to record one group's result.

1. Summary

Milestone	Pattern focus	Marks	Full marks require
M1	Command basics	20	Correct roles (8); ≥ 8 commands, NoCommand, WallPanel never names a vendor class (8); design answer (4)
M2	Stateful undo	20	Every command undoable (6); state captured in execute() (6); capped history stack works (4); design answer (4)
M3	Macros and logging	15	Variant mode as On and Off macros (5); reverse-order undo (4); audit log and replay (3); design answer (3)
M4	Adapter	20	Object adapter composes the adaptee (6); correct variant input mapping and errors (6); documented UnsupportedOperationException (3); design answer (5)
M5	Facade and least knowledge	25	Facade with correct ordering (8); facade commands on the panel (6); dashboard violation fixed (5); design answer (6)
Total		100	
Stretch	Two-way adapter	+10	One class implements both the legacy and new attendance interfaces; startLecture() marks the lecturer present

2. Detailed Criteria

Award the full mark, a partial mark, or little to none. Partial marks may be any value in between.

Criterion (marks)	Full	Partial	Little or none
M1 Roles (8)	Client, Invoker, Command and Receiver are clearly separated; WallPanel depends only on Command	Roles mostly right, but the panel or loader mixes responsibilities	No real separation; buttons call vendor methods directly
M1 Commands and NoCommand (8)	≥ 8 commands across ≥ 5 devices; NoCommand used; no null checks; readable toString()	Fewer commands, or null checks still present	Fewer than 4 commands, or does not compile
M1 Design answer (4)	Names the roles in their own code and explains why the panel imports no vendor class	Correct, but generic or textbook wording	Missing or wrong
M2 All commands undoable (6)	Every command, NoCommand included, has a correct undo()	Most commands undo correctly	Undo missing or broken
M2 State in execute() (6)	Volume, dim level and temperature are captured inside execute() and restored exactly	State captured, but in the wrong place or for only some devices	No previous state stored
M2 History stack (4)	Deque capped at the variant's depth; the oldest entry is dropped; demo empties the stack	Stack works but ignores the cap	Single undo only
M2 Design answer (4)	Gives a concrete failure case from their own hall	Correct, but abstract	Missing or wrong
M3 Variant mode macros (5)	The variant's exact steps, in order, as On and Off macros	Wrong mode, or steps missing	No MacroCommand

Criterion (marks)	Full	Partial	Little or none
M3 Reverse-order undo (4)	Macro undo iterates backwards, including when undone from the history	Undo runs forwards	No macro undo
M3 Audit log and replay (3)	Logs timestamp, slot and class; replay restores state on fresh devices	Logs, but replay is missing	No log
M3 Design answer (3)	Explains the PA on/volume ordering failure	Partially explained	Missing
M4 Object adapter (6)	Implements Projector and composes the variant's vendor class; ProjectorCheck is unchanged	Adapter works, but extends a class or changes ProjectorCheck	Not an adapter
M4 Mapping and errors (6)	All three sources map to the variant's codes; an unknown source throws IllegalArgumentException	Mapping partly wrong, or no validation	Hard-coded or another group's codes
M4 Unsupported method (3)	setKeystone() throws UnsupportedOperationException and is documented in Javadoc	Throws, but undocumented	Silently ignored
M4 Design answer (5)	Accurate on class adapters and Java's single inheritance; sensible choice for the subclass case	Partly accurate	Missing or wrong
M5 Facade ordering (8)	startLecture()/endLecture() and the mode methods run in a correct order and use the adapter	Works, but the order is wrong in places	Facade missing
M5 Facade commands on panel (6)	StartLectureCommand and EndLectureCommand use the facade as receiver, load into a slot, and undo	Commands exist, but bypass the facade or lack undo	Missing
M5 Least Knowledge fix (5)	CampusDashboard talks only to Building; the new delegating methods are added along the chain	Partly fixed	Unchanged
M5 Design answer (6)	A reasoned macro-versus-facade choice, using their own hall	Generic answer	Missing

3. Live Defence and Integrity

- Each group gets a **5-minute live defence**. Pick any member and ask them to explain one class, then make a small change live, e.g. “Add a BlindsOpenCommand to slot 2 and show undo working.”
- If the student can't explain or change their own submitted code, cap that milestone at **50%** and record why.
- Design answers that are generic textbook text, or identical across groups, score **zero** for that answer.
- A single “final commit” with no earlier history, or undisclosed AI use, is referred under the integrity policy.

4. Group Marking Sheet

Photocopy one per group.

Group: _____ Variant row: _____ Marker: _____ Date: _____

Milestone	Criterion	Max	Score	Comments
M1	Roles	8		

Milestone	Criterion	Max	Score	Comments
	Commands and NoCommand	8		
	Design answer	4		
M2	All commands undoable	6		
	State in execute()	6		
	History stack	4		
	Design answer	4		
M3	Variant mode macros	5		
	Reverse-order undo	4		
	Audit log and replay	3		
	Design answer	3		
M4	Object adapter	6		
	Mapping and errors	6		
	Unsupported method	3		
	Design answer	5		
M5	Facade ordering	8		
	Facade commands on panel	6		
	Least Knowledge fix	5		
	Design answer	6		
Total		**100**		
Stretch	Two-way attendance adapter	+10		
Defence	Explained and changed code live? Yes / Partly / No			

Appendix A: Group Variant Sheets

One page per group. Each sheet gives your group's projector vendor class (the adaptee), the exact input mapping your adapter must implement, your mode, and your WallPanel settings. **Work only from your own group's sheet.** The marker checks every submission against it.

Group	Vendor	Mode	Slots	Undo depth
1	VisioMax	Exam Mode	6	5
2	LumaBeam	Guest Lecture Mode	7	4
3	ClearCast	Video Screening Mode	5	6
4	PixelForge	Night Class Mode	8	3
5	VisioMax	Power-Save Mode	7	8
6	LumaBeam	Seminar Mode	6	5
7	ClearCast	Lab Demo Mode	8	7
8	PixelForge	Viva Mode	5	4
9	VisioMax	Guest Lecture Mode	8	10
10	LumaBeam	Exam Mode	5	6
11	ClearCast	Night Class Mode	7	3
12	PixelForge	Video Screening Mode	6	9
13	VisioMax	Seminar Mode	5	5
14	LumaBeam	Power-Save Mode	8	4
15	ClearCast	Viva Mode	6	8
16	PixelForge	Lab Demo Mode	7	6

Group 1 Variant Sheet

Item	Group 1
Projector vendor (adaptee)	**VisioMax**: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`
Adapter class you write	`VisioMaxProjectorAdapter implements Projector`
Input codes	HDMI1 = `1`, HDMI2 = `2`, WIRELESS = `7`
Mode (Milestones 3 and 5)	**Exam Mode**: facade methods `startExamMode()` and `endExamMode()`
WallPanel slots	**6** (0 to 5). Slot 5: Start/EndLectureCommand. Slot 4: your mode macros. Slots 0 to 3: device commands or NoCommand
Undo history depth	**5** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: VISIOMAXPROJECTOR.JAVA

```java
// Vendor: VisioMax. Closed source: you may NOT modify this class.
public class VisioMaxProjector {
    public void switchOn(int warmUpSeconds) {          // must be >= 5
        System.out.println("VisioMax warming up for " + warmUpSeconds + "s");
    }
    public void switchOff(boolean runCoolingFan) {     // pass true in a lecture hall
        System.out.println("VisioMax off (cooling fan: " + runCoolingFan + ")");
    }
    public void selectInput(int port) {   // Group 1: HDMI1=1, HDMI2=2, WIRELESS=7
        System.out.println("VisioMax input port " + port);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `switchOn(5)`
`off()`	call `switchOff(true)`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `selectInput(1)` / `selectInput(2)` / `selectInput(7)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `switchOn` needs a warm-up of **at least 5 seconds**. Always pass `true` to `switchOff` in a lecture hall so the cooling fan runs.

Your mode: Exam Mode

Macro	Steps (in this order)
On	Lights 100% → PA mute → blinds open → recorder stop → AC 21 °C → screen up
Off	Reverse it, or restore the previous state

Group members:

_____

Group 2 Variant Sheet

Item	Group 2
Projector vendor (adaptee)	**LumaBeam**: `powerUp()`, `powerDown()`, `routeTo(char)`
Adapter class you write	`LumaBeamProjectorAdapter implements Projector`
Input codes	HDMI1 = `'A'`, HDMI2 = `'B'`, WIRELESS = `'W'`
Mode (Milestones 3 and 5)	**Guest Lecture Mode**: facade methods `startGuestLectureMode()` and `endGuestLectureMode()`
WallPanel slots	**7** (0 to 6). Slot 6: Start/EndLectureCommand. Slot 5: your mode macros. Slots 0 to 4: device commands or NoCommand
Undo history depth	**4** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: LUMABEAMPROJECTOR.JAVA

```java
// Vendor: LumaBeam. Closed source: you may NOT modify this class.
public class LumaBeamProjector {
    public void powerUp()   { System.out.println("LumaBeam powering up"); }
    public void powerDown() { System.out.println("LumaBeam powering down"); }

    public void routeTo(char channel) {   // Group 2: HDMI1='A', HDMI2='B', WIRELESS='W'
        if ("ABW".indexOf(channel) < 0) return;   // unknown: silently ignored!
        System.out.println("LumaBeam routed to channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `powerUp()`
`off()`	call `powerDown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `routeTo('A')` / `routeTo('B')` / `routeTo('W')`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `routeTo` **silently ignores** characters it doesn't recognise, so your adapter must reject an unknown source itself.

Your mode: Guest Lecture Mode

Macro	Steps (in this order)
On	Lights 70% → screen down → projector on, HDMI2 → PA on, volume 6 → AC 23 °C → recorder start
Off	Recorder stop, then shut down

Group members:

_____

Group 3 Variant Sheet

Item	Group 3
Projector vendor (adaptee)	**ClearCast**: `start()`, `shutdown()`, `input(String)`
Adapter class you write	`ClearCastProjectorAdapter implements Projector`
Input codes	HDMI1 = `"H1"`, HDMI2 = `"H2"`, WIRELESS = `"CAST"`
Mode (Milestones 3 and 5)	**Video Screening Mode**: facade methods `startVideoScreeningMode()` and `endVideoScreeningMode()`
WallPanel slots	**5** (0 to 4). Slot 4: Start/EndLectureCommand. Slot 3: your mode macros. Slots 0 to 2: device commands or NoCommand
Undo history depth	**6** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: CLEARCASTPROJECTOR.JAVA

```java
// Vendor: ClearCast. Closed source: you may NOT modify this class.
public class ClearCastProjector {
    public void start()    { System.out.println("ClearCast starting"); }
    public void shutdown() { System.out.println("ClearCast shutting down"); }

    public void input(String code) {   // Group 3: HDMI1="H1", HDMI2="H2", WIRELESS="CAST"
        System.out.println("ClearCast input " + code);   // case-sensitive
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `start()`
`off()`	call `shutdown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `input("H1")` / `input("H2")` / `input("CAST")`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `input` codes are **case-sensitive**: `"h1"` is not `"H1"`.

Your mode: Video Screening Mode

Macro	Steps (in this order)
On	Blinds close → lights 10% → screen down → projector on, HDMI1 → PA on, volume 8
Off	Lights 100%, devices off, blinds open

Group members:

_____

Group 4 Variant Sheet

Item	Group 4
Projector vendor (adaptee)	**PixelForge**: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`
Adapter class you write	`PixelForgeProjectorAdapter implements Projector`
Input codes	HDMI1 = `10`, HDMI2 = `11`, WIRELESS = `20`
Mode (Milestones 3 and 5)	**Night Class Mode**: facade methods `startNightClassMode()` and `endNightClassMode()`
WallPanel slots	**8** (0 to 7). Slot 7: Start/EndLectureCommand. Slot 6: your mode macros. Slots 0 to 5: device commands or NoCommand
Undo history depth	**3** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: PIXELFORGEPROJECTOR.JAVA

```java
// Vendor: PixelForge. Closed source: you may NOT modify this class.
public class PixelForgeProjector {
    public void activate(boolean eco) {
        System.out.println("PixelForge on" + (eco ? " (eco mode)" : ""));
    }
    public void deactivate() { System.out.println("PixelForge off"); }

    public void useChannel(int channel) {   // Group 4: HDMI1=10, HDMI2=11, WIRELESS=20
        System.out.println("PixelForge channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `activate(false)`
`off()`	call `deactivate()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `useChannel(10)` / `useChannel(11)` / `useChannel(20)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `activate(true)` switches on **eco mode**, which is too dim for a lecture hall, so always call `activate(false)`.

Your mode: Night Class Mode

Macro	Steps (in this order)
On	Lights 100% → blinds close → AC 24 °C → PA on, volume 5 → projector on, WIRELESS
Off	All off; blinds stay closed

Group members:

_____

Group 5 Variant Sheet

Item	Group 5
Projector vendor (adaptee)	**VisioMax**: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`
Adapter class you write	`VisioMaxProjectorAdapter implements Projector`
Input codes	HDMI1 = `3`, HDMI2 = `4`, WIRELESS = `9`
Mode (Milestones 3 and 5)	**Power-Save Mode**: facade methods `startPowerSaveMode()` and `endPowerSaveMode()`
WallPanel slots	**7** (0 to 6). Slot 6: Start/EndLectureCommand. Slot 5: your mode macros. Slots 0 to 4: device commands or NoCommand
Undo history depth	**8** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: VISIOMAXPROJECTOR.JAVA

```java
// Vendor: VisioMax. Closed source: you may NOT modify this class.
public class VisioMaxProjector {
    public void switchOn(int warmUpSeconds) {          // must be >= 5
        System.out.println("VisioMax warming up for " + warmUpSeconds + "s");
    }
    public void switchOff(boolean runCoolingFan) {     // pass true in a lecture hall
        System.out.println("VisioMax off (cooling fan: " + runCoolingFan + ")");
    }
    public void selectInput(int port) {   // Group 5: HDMI1=3, HDMI2=4, WIRELESS=9
        System.out.println("VisioMax input port " + port);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `switchOn(5)`
`off()`	call `switchOff(true)`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `selectInput(3)` / `selectInput(4)` / `selectInput(9)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `switchOn` needs a warm-up of **at least 5 seconds**. Always pass `true` to `switchOff` in a lecture hall so the cooling fan runs.

Your mode: Power-Save Mode

Macro	Steps (in this order)
On	Projector off → PA off → AC off → screen up → lights off
Off	Restore the previous state of every device

Group members:

_____

Group 6 Variant Sheet

Item	Group 6
Projector vendor (adaptee)	**LumaBeam**: `powerUp()`, `powerDown()`, `routeTo(char)`
Adapter class you write	`LumaBeamProjectorAdapter implements Projector`
Input codes	HDMI1 = `'X'`, HDMI2 = `'Y'`, WIRELESS = `'Z'`
Mode (Milestones 3 and 5)	**Seminar Mode**: facade methods `startSeminarMode()` and `endSeminarMode()`
WallPanel slots	**6** (0 to 5). Slot 5: Start/EndLectureCommand. Slot 4: your mode macros. Slots 0 to 3: device commands or NoCommand
Undo history depth	**5** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: LUMABEAMPROJECTOR.JAVA

```java
// Vendor: LumaBeam. Closed source: you may NOT modify this class.
public class LumaBeamProjector {
    public void powerUp()   { System.out.println("LumaBeam powering up"); }
    public void powerDown() { System.out.println("LumaBeam powering down"); }

    public void routeTo(char channel) {   // Group 6: HDMI1='X', HDMI2='Y', WIRELESS='Z'
        if ("XYZ".indexOf(channel) < 0) return;   // unknown: silently ignored!
        System.out.println("LumaBeam routed to channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `powerUp()`
`off()`	call `powerDown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `routeTo('X')` / `routeTo('Y')` / `routeTo('Z')`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `routeTo` **silently ignores** characters it doesn't recognise, so your adapter must reject an unknown source itself.

Your mode: Seminar Mode

Macro	Steps (in this order)
On	Blinds open → lights 80% → PA on, volume 4 → projector on, WIRELESS → recorder start
Off	Recorder stop, then shut down

Group members:

_____

Group 7 Variant Sheet

Item	Group 7
Projector vendor (adaptee)	**ClearCast**: `start()`, `shutdown()`, `input(String)`
Adapter class you write	`ClearCastProjectorAdapter implements Projector`
Input codes	HDMI1 = `"IN-1"`, HDMI2 = `"IN-2"`, WIRELESS = `"AIR"`
Mode (Milestones 3 and 5)	**Lab Demo Mode**: facade methods `startLabDemoMode()` and `endLabDemoMode()`
WallPanel slots	**8** (0 to 7). Slot 7: Start/EndLectureCommand. Slot 6: your mode macros. Slots 0 to 5: device commands or NoCommand
Undo history depth	**7** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: CLEARCASTPROJECTOR.JAVA

```java
// Vendor: ClearCast. Closed source: you may NOT modify this class.
public class ClearCastProjector {
    public void start()    { System.out.println("ClearCast starting"); }
    public void shutdown() { System.out.println("ClearCast shutting down"); }

    public void input(String code) {   // Group 7: HDMI1="IN-1", HDMI2="IN-2", WIRELESS="AIR"
        System.out.println("ClearCast input " + code);   // case-sensitive
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `start()`
`off()`	call `shutdown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `input("IN-1")` / `input("IN-2")` / `input("AIR")`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `input` codes are **case-sensitive**: `"h1"` is not `"H1"`.

Your mode: Lab Demo Mode

Macro	Steps (in this order)
On	Lights 60% → screen down → projector on, HDMI2 → PA on, volume 7 → AC 22 °C
Off	Reverse it

Group members:

_____

Group 8 Variant Sheet

Item	Group 8
Projector vendor (adaptee)	**PixelForge**: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`
Adapter class you write	`PixelForgeProjectorAdapter implements Projector`
Input codes	HDMI1 = `1`, HDMI2 = `2`, WIRELESS = `99`
Mode (Milestones 3 and 5)	**Viva Mode**: facade methods `startVivaMode()` and `endVivaMode()`
WallPanel slots	**5** (0 to 4). Slot 4: Start/EndLectureCommand. Slot 3: your mode macros. Slots 0 to 2: device commands or NoCommand
Undo history depth	**4** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: PIXELFORGEPROJECTOR.JAVA

```java
// Vendor: PixelForge. Closed source: you may NOT modify this class.
public class PixelForgeProjector {
    public void activate(boolean eco) {
        System.out.println("PixelForge on" + (eco ? " (eco mode)" : ""));
    }
    public void deactivate() { System.out.println("PixelForge off"); }

    public void useChannel(int channel) {   // Group 8: HDMI1=1, HDMI2=2, WIRELESS=99
        System.out.println("PixelForge channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `activate(false)`
`off()`	call `deactivate()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `useChannel(1)` / `useChannel(2)` / `useChannel(99)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `activate(true)` switches on **eco mode**, which is too dim for a lecture hall, so always call `activate(false)`.

Your mode: Viva Mode

Macro	Steps (in this order)
On	Blinds close → lights 90% → PA on, volume 3 → recorder start → screen up
Off	Recorder stop, PA off, lights 100%

Group members:

_____

Group 9 Variant Sheet

Item	Group 9
Projector vendor (adaptee)	**VisioMax**: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`
Adapter class you write	`VisioMaxProjectorAdapter implements Projector`
Input codes	HDMI1 = `5`, HDMI2 = `6`, WIRELESS = `8`
Mode (Milestones 3 and 5)	**Guest Lecture Mode**: facade methods `startGuestLectureMode()` and `endGuestLectureMode()`
WallPanel slots	**8** (0 to 7). Slot 7: Start/EndLectureCommand. Slot 6: your mode macros. Slots 0 to 5: device commands or NoCommand
Undo history depth	**10** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: VISIOMAXPROJECTOR.JAVA

```java
// Vendor: VisioMax. Closed source: you may NOT modify this class.
public class VisioMaxProjector {
    public void switchOn(int warmUpSeconds) {          // must be >= 5
        System.out.println("VisioMax warming up for " + warmUpSeconds + "s");
    }
    public void switchOff(boolean runCoolingFan) {     // pass true in a lecture hall
        System.out.println("VisioMax off (cooling fan: " + runCoolingFan + ")");
    }
    public void selectInput(int port) {   // Group 9: HDMI1=5, HDMI2=6, WIRELESS=8
        System.out.println("VisioMax input port " + port);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `switchOn(5)`
`off()`	call `switchOff(true)`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `selectInput(5)` / `selectInput(6)` / `selectInput(8)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `switchOn` needs a warm-up of **at least 5 seconds**. Always pass `true` to `switchOff` in a lecture hall so the cooling fan runs.

Your mode: Guest Lecture Mode

Macro	Steps (in this order)
On	Lights 70% → screen down → projector on, HDMI2 → PA on, volume 6 → AC 23 °C → recorder start
Off	Recorder stop, then shut down

Group members:

_____

Group 10 Variant Sheet

Item	Group 10
Projector vendor (adaptee)	**LumaBeam**: `powerUp()`, `powerDown()`, `routeTo(char)`
Adapter class you write	`LumaBeamProjectorAdapter implements Projector`
Input codes	HDMI1 = `'1'`, HDMI2 = `'2'`, WIRELESS = `'R'`
Mode (Milestones 3 and 5)	**Exam Mode**: facade methods `startExamMode()` and `endExamMode()`
WallPanel slots	**5** (0 to 4). Slot 4: Start/EndLectureCommand. Slot 3: your mode macros. Slots 0 to 2: device commands or NoCommand
Undo history depth	**6** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: LUMABEAMPROJECTOR.JAVA

```java
// Vendor: LumaBeam. Closed source: you may NOT modify this class.
public class LumaBeamProjector {
    public void powerUp()   { System.out.println("LumaBeam powering up"); }
    public void powerDown() { System.out.println("LumaBeam powering down"); }

    public void routeTo(char channel) {   // Group 10: HDMI1='1', HDMI2='2', WIRELESS='R'
        if ("12R".indexOf(channel) < 0) return;   // unknown: silently ignored!
        System.out.println("LumaBeam routed to channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `powerUp()`
`off()`	call `powerDown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `routeTo('1')` / `routeTo('2')` / `routeTo('R')`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `routeTo` **silently ignores** characters it doesn't recognise, so your adapter must reject an unknown source itself.

Your mode: Exam Mode

Macro	Steps (in this order)
On	Lights 100% → PA mute → blinds open → recorder stop → AC 21 °C → screen up
Off	Reverse it, or restore the previous state

Group members:

_____

Group 11 Variant Sheet

Item	Group 11
Projector vendor (adaptee)	**ClearCast**: `start()`, `shutdown()`, `input(String)`
Adapter class you write	`ClearCastProjectorAdapter implements Projector`
Input codes	HDMI1 = `"A"`, HDMI2 = `"B"`, WIRELESS = `"MIRROR"`
Mode (Milestones 3 and 5)	**Night Class Mode**: facade methods `startNightClassMode()` and `endNightClassMode()`
WallPanel slots	**7** (0 to 6). Slot 6: Start/EndLectureCommand. Slot 5: your mode macros. Slots 0 to 4: device commands or NoCommand
Undo history depth	**3** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: CLEARCASTPROJECTOR.JAVA

```java
// Vendor: ClearCast. Closed source: you may NOT modify this class.
public class ClearCastProjector {
    public void start()    { System.out.println("ClearCast starting"); }
    public void shutdown() { System.out.println("ClearCast shutting down"); }

    public void input(String code) {   // Group 11: HDMI1="A", HDMI2="B", WIRELESS="MIRROR"
        System.out.println("ClearCast input " + code);   // case-sensitive
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `start()`
`off()`	call `shutdown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `input("A")` / `input("B")` / `input("MIRROR")`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `input` codes are **case-sensitive**: `"h1"` is not `"H1"`.

Your mode: Night Class Mode

Macro	Steps (in this order)
On	Lights 100% → blinds close → AC 24 °C → PA on, volume 5 → projector on, WIRELESS
Off	All off; blinds stay closed

Group members:

_____

Group 12 Variant Sheet

Item	Group 12
Projector vendor (adaptee)	**PixelForge**: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`
Adapter class you write	`PixelForgeProjectorAdapter implements Projector`
Input codes	HDMI1 = `30`, HDMI2 = `31`, WIRELESS = `40`
Mode (Milestones 3 and 5)	**Video Screening Mode**: facade methods `startVideoScreeningMode()` and `endVideoScreeningMode()`
WallPanel slots	**6** (0 to 5). Slot 5: Start/EndLectureCommand. Slot 4: your mode macros. Slots 0 to 3: device commands or NoCommand
Undo history depth	**9** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: PIXELFORGEPROJECTOR.JAVA

```java
// Vendor: PixelForge. Closed source: you may NOT modify this class.
public class PixelForgeProjector {
    public void activate(boolean eco) {
        System.out.println("PixelForge on" + (eco ? " (eco mode)" : ""));
    }
    public void deactivate() { System.out.println("PixelForge off"); }

    public void useChannel(int channel) {   // Group 12: HDMI1=30, HDMI2=31, WIRELESS=40
        System.out.println("PixelForge channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `activate(false)`
`off()`	call `deactivate()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `useChannel(30)` / `useChannel(31)` / `useChannel(40)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `activate(true)` switches on **eco mode**, which is too dim for a lecture hall, so always call `activate(false)`.

Your mode: Video Screening Mode

Macro	Steps (in this order)
On	Blinds close → lights 10% → screen down → projector on, HDMI1 → PA on, volume 8
Off	Lights 100%, devices off, blinds open

Group members:

_____

Group 13 Variant Sheet

Item	Group 13
Projector vendor (adaptee)	**VisioMax**: `switchOn(int)`, `switchOff(boolean)`, `selectInput(int)`
Adapter class you write	`VisioMaxProjectorAdapter implements Projector`
Input codes	HDMI1 = `2`, HDMI2 = `1`, WIRELESS = `0`
Mode (Milestones 3 and 5)	**Seminar Mode**: facade methods `startSeminarMode()` and `endSeminarMode()`
WallPanel slots	**5** (0 to 4). Slot 4: Start/EndLectureCommand. Slot 3: your mode macros. Slots 0 to 2: device commands or NoCommand
Undo history depth	**5** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: VISIOMAXPROJECTOR.JAVA

```java
// Vendor: VisioMax. Closed source: you may NOT modify this class.
public class VisioMaxProjector {
    public void switchOn(int warmUpSeconds) {          // must be >= 5
        System.out.println("VisioMax warming up for " + warmUpSeconds + "s");
    }
    public void switchOff(boolean runCoolingFan) {     // pass true in a lecture hall
        System.out.println("VisioMax off (cooling fan: " + runCoolingFan + ")");
    }
    public void selectInput(int port) {   // Group 13: HDMI1=2, HDMI2=1, WIRELESS=0
        System.out.println("VisioMax input port " + port);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `switchOn(5)`
`off()`	call `switchOff(true)`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `selectInput(2)` / `selectInput(1)` / `selectInput(0)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `switchOn` needs a warm-up of **at least 5 seconds**. Always pass `true` to `switchOff` in a lecture hall so the cooling fan runs.

Your mode: Seminar Mode

Macro	Steps (in this order)
On	Blinds open → lights 80% → PA on, volume 4 → projector on, WIRELESS → recorder start
Off	Recorder stop, then shut down

Group members:

_____

Group 14 Variant Sheet

Item	Group 14
Projector vendor (adaptee)	**LumaBeam**: `powerUp()`, `powerDown()`, `routeTo(char)`
Adapter class you write	`LumaBeamProjectorAdapter implements Projector`
Input codes	HDMI1 = `'P'`, HDMI2 = `'Q'`, WIRELESS = `'S'`
Mode (Milestones 3 and 5)	**Power-Save Mode**: facade methods `startPowerSaveMode()` and `endPowerSaveMode()`
WallPanel slots	**8** (0 to 7). Slot 7: Start/EndLectureCommand. Slot 6: your mode macros. Slots 0 to 5: device commands or NoCommand
Undo history depth	**4** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: LUMABEAMPROJECTOR.JAVA

```java
// Vendor: LumaBeam. Closed source: you may NOT modify this class.
public class LumaBeamProjector {
    public void powerUp()   { System.out.println("LumaBeam powering up"); }
    public void powerDown() { System.out.println("LumaBeam powering down"); }

    public void routeTo(char channel) {   // Group 14: HDMI1='P', HDMI2='Q', WIRELESS='S'
        if ("PQS".indexOf(channel) < 0) return;   // unknown: silently ignored!
        System.out.println("LumaBeam routed to channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `powerUp()`
`off()`	call `powerDown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `routeTo('P')` / `routeTo('Q')` / `routeTo('S')`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `routeTo` **silently ignores** characters it doesn't recognise, so your adapter must reject an unknown source itself.

Your mode: Power-Save Mode

Macro	Steps (in this order)
On	Projector off → PA off → AC off → screen up → lights off
Off	Restore the previous state of every device

Group members:

_____

Group 15 Variant Sheet

Item	Group 15
Projector vendor (adaptee)	**ClearCast**: `start()`, `shutdown()`, `input(String)`
Adapter class you write	`ClearCastProjectorAdapter implements Projector`
Input codes	HDMI1 = `"PORT_A"`, HDMI2 = `"PORT_B"`, WIRELESS = `"WIFI"`
Mode (Milestones 3 and 5)	**Viva Mode**: facade methods `startVivaMode()` and `endVivaMode()`
WallPanel slots	**6** (0 to 5). Slot 5: Start/EndLectureCommand. Slot 4: your mode macros. Slots 0 to 3: device commands or NoCommand
Undo history depth	**8** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: CLEARCASTPROJECTOR.JAVA

```java
// Vendor: ClearCast. Closed source: you may NOT modify this class.
public class ClearCastProjector {
    public void start()    { System.out.println("ClearCast starting"); }
    public void shutdown() { System.out.println("ClearCast shutting down"); }

    public void input(String code) {   // Group 15: HDMI1="PORT_A", HDMI2="PORT_B", WIRELESS="WIFI"
        System.out.println("ClearCast input " + code);   // case-sensitive
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `start()`
`off()`	call `shutdown()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `input("PORT_A")` / `input("PORT_B")` / `input("WIFI")`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `input` codes are **case-sensitive**: `"h1"` is not `"H1"`.

Your mode: Viva Mode

Macro	Steps (in this order)
On	Blinds close → lights 90% → PA on, volume 3 → recorder start → screen up
Off	Recorder stop, PA off, lights 100%

Group members:

_____

Group 16 Variant Sheet

Item	Group 16
Projector vendor (adaptee)	**PixelForge**: `activate(boolean eco)`, `deactivate()`, `useChannel(int)`
Adapter class you write	`PixelForgeProjectorAdapter implements Projector`
Input codes	HDMI1 = `7`, HDMI2 = `8`, WIRELESS = `15`
Mode (Milestones 3 and 5)	**Lab Demo Mode**: facade methods `startLabDemoMode()` and `endLabDemoMode()`
WallPanel slots	**7** (0 to 6). Slot 6: Start/EndLectureCommand. Slot 5: your mode macros. Slots 0 to 4: device commands or NoCommand
Undo history depth	**6** commands (drop the oldest when full)

Your adaptee: starter code (copy it as given)

JAVA: PIXELFORGEPROJECTOR.JAVA

```java
// Vendor: PixelForge. Closed source: you may NOT modify this class.
public class PixelForgeProjector {
    public void activate(boolean eco) {
        System.out.println("PixelForge on" + (eco ? " (eco mode)" : ""));
    }
    public void deactivate() { System.out.println("PixelForge off"); }

    public void useChannel(int channel) {   // Group 16: HDMI1=7, HDMI2=8, WIRELESS=15
        System.out.println("PixelForge channel " + channel);
    }
    // NOTE: no keystone support
}
```

Milestone 4: required adapter mapping

When the client calls…	your adapter must…
`on()`	call `activate(false)`
`off()`	call `deactivate()`
`setSource("HDMI1")` / `"HDMI2"` / `"WIRELESS"`	call `useChannel(7)` / `useChannel(8)` / `useChannel(15)`
`setSource("HDMI3")` or any other value	throw `IllegalArgumentException` with a clear message
`setKeystone(anything)`	throw `UnsupportedOperationException`, documented in Javadoc

Vendor quirk: `activate(true)` switches on **eco mode**, which is too dim for a lecture hall, so always call `activate(false)`.

Your mode: Lab Demo Mode

Macro	Steps (in this order)
On	Lights 60% → screen down → projector on, HDMI2 → PA on, volume 7 → AC 22 °C
Off	Reverse it

Group members:

_____